Week3 Mini Project

Section A - Data Engineering Theory

Q1. OLTP vs OLAP Architecture

The company currently stores transactional and analytical workloads in the same SQL database.

Tasks

1. Explain why this creates performance issues.
2. Compare OLTP and OLAP workloads.
3. Suggest an improved architecture.
4. Explain where:
    - PostgreSQL /SQLServer/MySql
    - Data warehouse
    - Data lake

should be used.

Solution:

1. **OLTP** handles real-time transactional data. Examples would be placing an order, transfer money, update a profile and so on, whereas **OLAP** handles large historical dataset for analysis.

    Storing both the OLAP and OLTP in the same SQL database would cause in increased latency, since OLTP prioritizes database for reads to return small amount of data efficiently. On the other hand, OLAP queries are complex , works across millions of data, compute aggregation across longer time ranges.

    OLTP transactions such as INERT, UPDATE, DELETE should happen instantaneously, whereas OLAP queries can take few seconds to respond. Imagine if a customer places an order, the transaction should occur immediately. At the same time, an Analyst runs the query scanning over millions of data. This where the conflict arises.

    A **lock** protects data while it is being accessed or changed. If a transaction occurs for updating a selected row. The database may lock that row while the update happens. OLTP operations normally touch very few rows, so locks tend to be small and short-lived. But an analytical query might access enormous sections of a table. Depending on the database

and isolation mechanism, this can create contention between readers and writers. This is known as **resource contention**.

Lastly, OLAP queries consume most of the available CPU, causing the transactional queries to wait for CPU or disk access.

2.

Features	OLTP	OLAP
Primary purpose	Managing real-time operational transactions (e.g., e-commerce checkouts, bank transfers).	Analyzing large volumes of historical and aggregated data for business intelligence.
Data Flow	High-frequency writes, inserts, updates, and deletes.	Read-intensive queries, scans, and aggregations.
Response Time	Milliseconds (sub-50ms) for point lookups.	Seconds to minutes or hours depending on data scale.
Data Storage	Row-oriented storage (optimized for single-row access).	Columnar storage (optimized for compression and multi-column scans).
Data Schema	Normalized schemas (to remove redundancy and ensure integrity).	Denormalized or dimensional schemas like star/snowflake.
Consistency	Strict ACID guarantees.	Relaxed or eventual consistency

3. An improved architecture would be to store the transactional data in the Database and to move the data after retaining it for a period in the Datawarehouse as historical dataset.

4.

- PostgreSQL /SQLServer/MySql: It should be used for transactional data
- Data warehouse: To store structured historical dataset
- Data lake: to store structured, semi-structured, and unstructured data, both raw and processed

Q2. ACID vs BASE in Distributed Systems

QuickCart wants to store clickstream events from millions of users.

Tasks

1. Explain why BASE systems are preferred for clickstream systems.

2. Which workloads require ACID compliance?

3. Give examples where eventual consistency is acceptable.

Solution:

1. BASE consistency means that the system is designed to be Basically Available and to keep responding to requests even though some components are temporarily missing. As synchronization occurs across the distributed systems the state changes or is in transition. The replicas eventually agree on the data after no new updates and reconciling the pending update.

   Therefore, clickstream events have BASE consistency because it prioritize **high availability and speed** over perfect, real-time accuracy.

   Clickstream data involves capturing millions of actions (clicks, hovers, page views) per second. **BASE systems** allow the application to write data instantly without waiting for confirmation from the rest of the database cluster.

   If a database node goes down or network latency spikes, a clickstream system must keep recording user actions. It is much better to capture a user's click slightly out of order or with a slight delay than to crash the webpage or drop the tracking event entirely. The system remains **available** to accept data at all costs.

2. Since Quickmart is an e-commerce and logistics platform, data such as:

   a. order processing

   b. inventory managment

   c. payment and refunds

   should follow strict ACID.

3. Clickstream events or product ratings and reviews can have eventual consistency where the data eventually appears in the analytical system.

Q3. Storage Formats & Querying

Tasks

1. Compare:

- CSV
- JSON
- Avro
- Parquet

2. Why do analytical systems prefer columnar storage?

3. Explain avro vs json format.

4. Explain compression benefits in Parquet.

Solution:

1.

- **CSV**: It is the most widely used file format due to its ease of use and widely supported systems. The data are stored in Tabular format with comma-separated. Some of its features are:
    - Human-readable.
    - Row-based storage.
    - Great for small datasets, quick exports, sharing with non-technical people.

    Limitations:
    - Cannot store complex structures such as hierarchical data or key-value pair.
    - Since the data is stored in row-oriented it makes the queries slower as compared to files with column-oriented storage.
    - Not suitable for large datasets.
    - Not suitable for partitioned data storage.
- **JSON**: It is a semi-structured file format and stores data in key-value pairs, which is easy to read, understand, and supports hierarchical data. Some of its features are:
    - Human-readable.
    - Row-based storage but supports nesting.
    - Suitable for data object exchange.

    Limitations:
    - Larger file size compared to CSV and Parquet.
    - Slower to scan than binary formats.
- **Parquet**: It is a binary columnar format built for analytics on big data and is the default choice in modern lakes and lakehouses. It has a built in metadata that stores schema and structure information. This makes it very beneficial for systems that use partitioned data storage. When reading data, the database does not need to read the entire file to determine if the queried data is within it, rather it only needs to access the metadata of the Parquet file to know immediately. Some of its key features are:
    - Small file size.
    - Suitable for partitioned data.
    - Fast read operations.
    - supports nested data.
    - stores data types.
    - ideal for large datasets

- Highly efficient data compression and decompression.
- Used for OLAP use cases

Limitations:

- Cannot be read with text editors, since it is in Binary.
- Limited support outside Apache Spark and Hadoop.
- Requires specialized knowledge.

- **AVRO**: It is a binary row-based format designed for efficiently moving data between systems- especially in streaming pipelines. It stores the data's schema along with the data as JSON, so the receiving system always knows the exact shape. Although Avro format stores data in a row-oriented structure, it has high read and write speeds because the data is serialized, meaning the data is converted from an in-memory object or structure into a format that can be stored in a file or transmitted between systems. Avro uses a schema to encode complete records efficiently in a compact binary format. some of its features are:
  - Stores Metadata in JSON.
  - Supports schema changes (allowing old and new data structures to coexist without breaking the application). Here the original stored record is not rewritten. The default is supplied while reading it.

  Limitations:

  - Weak at column-focused analytics.

2. Analytical system prefers columnar storage because it keeps all the values of one column together, rather than keeping each row's fields together. This lets analytical queries read only the columns they need and compress data more effectively, which is why columnar formats like Parquet are much faster for analytics than row-based formats. This way of storage has translated into hardware savings and minimized latency for accessing data.
3. Avro vs. JSON format:

Feature	Avro	JSON
Readability	No (binary)	Yes
Compression	Good	Poor
Best for	Streaming, transport data	APIs, app data
Query speed	Moderate	Slow

4. Compression in parquet files reduces storage drastically by 75 to 90% compared to row-based formats such as CSV, while boosting analytical query speed.

Q4. Partitioning, Indexing

Tasks

1. Design a partitioning strategy for:

-orders table

-clickstream data

2. Suggest indexing strategy for:

-customer lookups

-order filtering

-joins

3. Explain when indexes negatively affect performance.

Solution:

1. Data partitioning is a technique that involves dividing large tables or indexes into smaller, manageable subsets called partitions. Partitioning can be done based on various criteria, such as range partitioning (based on a specific range of values), list partitioning (based on predefined lists), hash partitioning (based on hash values), or composite partitioning (a combination of multiple partitioning techniques).

   For orders the data can be partitioned based on the year or the month so that the data for each year of the month is visible.

   For clickstream data partitioning it by event_timestamp.

   Clickstream analytics are typically time-based:

   - How many page views happened yesterday?
   - How many users visited QuickCart last week?
   - What was the conversion rate this month?
   - How many mobile sessions occurred today?

2. Indexing strategy for:
   a. customer lookups: with customer_id
   b. order filtering: with order_date, customer_id, payment_status
   c. joins: index foreign-key columns
3. Scenarios when the indexes affect the performance negatively:
   a. When the dataset is large , adding multiple indexes to a table can quickly use a significant amount of additional space.

b. When an index is added, it has to be updated every time a row is added, updated, or deleted. This means writes will be slower.

Q4. Database Design

Design schema for:

1. Shipment tracking

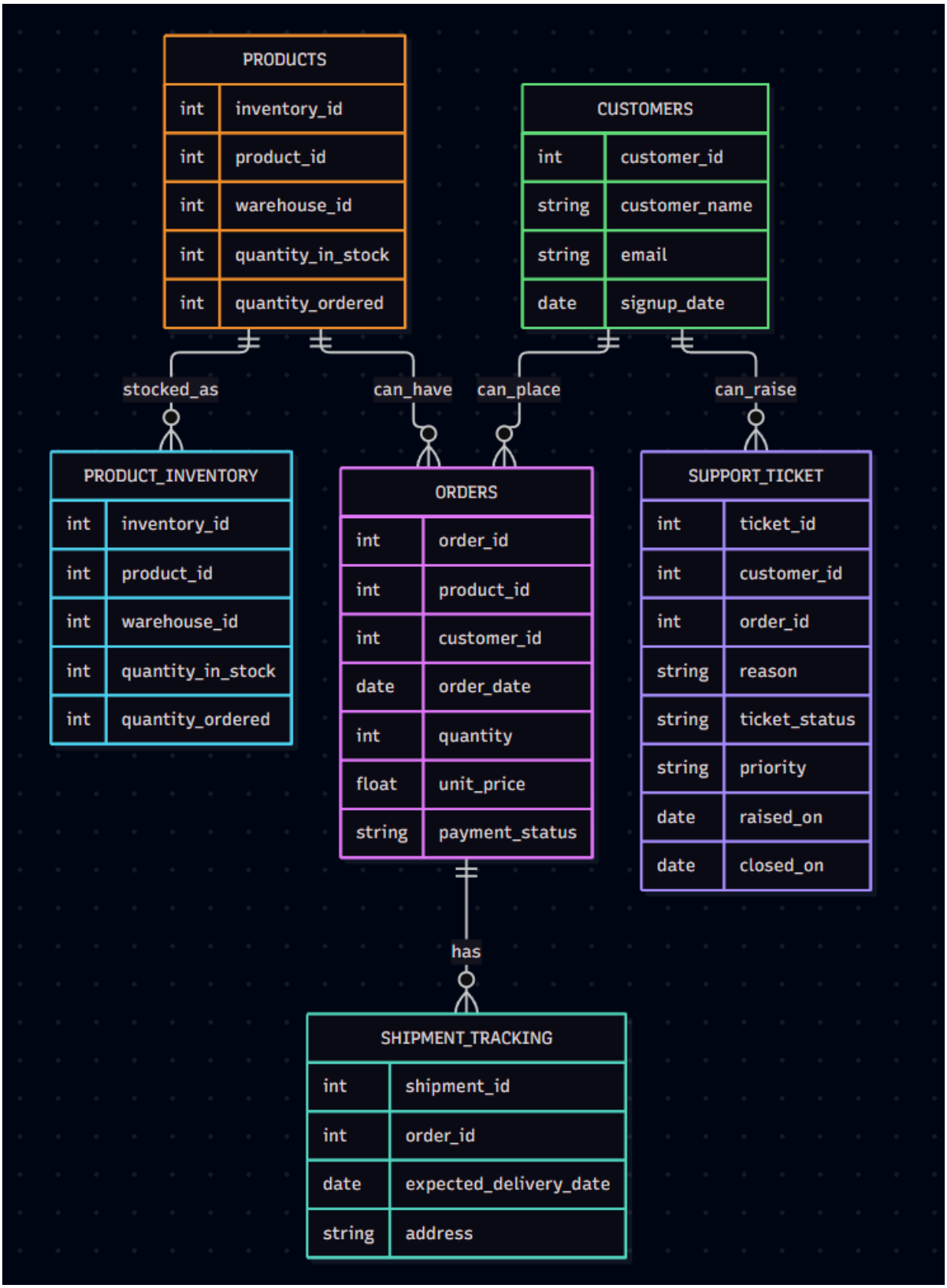
2. Product inventory

3. Customer support tickets

Requirements:

- normalized design
- primary keys
- foreign keys
- indexing recommendations

Solution:


PRODUCTS
int inventory_id
int product_id
int warehouse_id
int quantity_in_stock
int quantity_ordered
CUSTOMERS
int customer_id
string customer_name
string email
date signup_date
stocked_as
can_have
can_place
can_raise
PRODUCT_INVENTORY
int inventory_id
int product_id
int warehouse_id
int quantity_in_stock
int quantity_ordered
ORDERS
int order_id
int product_id
int customer_id
date order_date
int quantity
float unit_price
string payment_status
SUPPORT_TICKET
int ticket_id
int customer_id
int order_id
string reason
string ticket_status
string priority
date raised_on
date closed_on
has
SHIPMENT_TRACKING
int shipment_id
int order_id
date expected_delivery_date
string address